

BriefCheck: an assignment checklist backed by evidence

PA#1 | CSC13114 | Proposal and planning | Draft for review

Team: [Name A, student ID] (product/evaluation); [Name B, student ID] (application); [Name C, student ID] (LLM/testing). Maximum three members; for a solo submission, assign all work to that named student.

Repository: [Insert repository URL]

Planning assumptions: BriefCheck is a proposed idea, not a confirmed team choice. The six checkpoint labels and dates below are provisional internal targets, not verified course deadlines. Confirm them against the course calendar and replace all bracketed fields before submission.

1. Product, users, and problem

Our primary user is a CSC13114 student preparing a PA#1 submission, with an assignment brief and rubric open shortly before uploading the final ZIP. The problem is that submission requirements are scattered across prose, tables, and notes, so the student can finish the project while overlooking a required deliverable.

Today, the student rereads the documents and manually copies requirements into notes. BriefCheck is a small web application that turns an uploaded brief and rubric into one editable checklist, with the original passage beside each item. The student checks, corrects, and exports it before submitting through the existing course system. Before committing to this idea, we will observe three classmates completing that task and check whether the checklist saves review time without increasing omissions.

2. One LLM feature and the cost of being wrong

The single LLM feature is **extracting actionable submission requirements from assignment documents**, including conditions such as individual versus group submission. Each proposed item includes a source quote and page reference; ambiguous wording is flagged for review. For example, the supplied PA#1 rubric should yield an item to include SELF_ASSESSMENT_REPORT.md, with evidence from the submission section. The feature interprets varied wording and table context that fixed keyword matching may miss.

An omitted or misinterpreted requirement harms the student who trusts the checklist. In this rubric, omitting the self-assessment report incurs a 10-point deduction on the 100-point milestone scale. At a 4% course weight, that is 0.4 percentage points of the final course grade, assuming direct weighting. Before submission, the student can correct the checklist and add the file; after the deadline, recovery depends on the course's resubmission policy. An invented requirement can also waste work; pilot sessions will record unnecessary items and time spent correcting them rather than assume a time cost.

We will compare extracted items against a manually reviewed reference checklist, recording missed critical requirements, unsupported items, and incorrect conditions. Exact quote checks can detect invented evidence, but cannot prove the checklist is complete. The source remains visible, every generated item starts unreviewed, and we will make no claim that a checklist guarantees full compliance or a grade.

3. Semester scope

In: one English assignment brief plus one rubric as text-based PDFs; extraction of deliverables, submission rules, and stated dates; checklist items with quotes and page references; student editing and completion marks; saving a checklist; Markdown export; and evaluation on ten permitted briefs/rubrics with varied tables and conditional rules. A small hosted demo will support the complete upload-to-export workflow.

Out: writing assignment answers, grading submissions, automatic submission, LMS integration, calendar reminders, collaboration, multilingual documents, OCR for scanned pages, arbitrary web browsing, and fine-tuning. We will not resolve contradictory instructor instructions automatically. The pilot will use course materials we have permission to process, without student submissions or personal records.

4. Six-checkpoint plan and ownership

Dates are proposed targets in 2026. Each owner field must be replaced with one actual team member's name. Work and exit evidence are planning commitments; detailed specifications come later.

Checkpoint	Target date	Accountable owner	Work and evidence for review
1. Proposal and feasibility	8 Oct 2026	[Name A]	Interview three classmates; confirm permitted documents; submit proposal and repository link. Record whether the problem warrants building.
2. Core-feature specification	22 Oct 2026	[Name C]	Write the later feature spec; manually review ten document pairs; compare a keyword baseline with an LLM prototype. Record critical misses and unsupported items.
3. End-to-end prototype	5 Nov 2026	[Name B]	Connect upload, PDF text extraction, model call, and quote-linked checklist. Demonstrate one real document pair end to end.
4. Usable alpha	19 Nov 2026	[Name B]	Add edit, save, completion marks, and export. Test the workflow with three classmates and record blockers and correction time.
5. Reliability and evaluation	3 Dec 2026	[Name C]	Evaluate on held-out documents; test missing evidence, ambiguous rules, API failure, and token limits. Publish errors, latency, and measured cost.
6. Final delivery	17 Dec 2026	[Name A]	Fix critical pilot failures; deploy the demo; verify setup from a fresh clone; deliver demonstration and evaluation report with limitations.

5. Two risks that could sink the project

Risk 1: a plausible checklist hides a critical omission. If students must reread everything to catch frequent misses, the product provides little value. This week, label requirements in three permitted briefs and test extraction, especially footnotes, tables, and group conditions. Use visible evidence and human review; expand to ten pairs before the alpha. If critical omissions persist on held-out documents, narrow supported document layouts and report the limitation instead of presenting the tool as dependable.

Risk 2: document extraction corrupts the evidence. Flattened tables can pair a rule with the wrong condition or page; scanned PDFs may have no usable text. This week, test the PDF parser on three real documents, inspect extracted tables and page boundaries, and secure ten permitted evaluation pairs. Limit the semester to text-based PDFs; reject unusable extraction with an explanation. If table extraction remains unreliable, narrow the pilot to text layouts we can preserve and verify.

6. Technology choices and operating budget

- **Next.js with TypeScript:** one web application for upload, checklist editing, and server-side model calls keeps the semester deployment small and the API key off the client.
- **PDF.js:** extract text per page from the supported PDFs so checklist evidence can be traced back to the original document.
- **Supabase Postgres and Auth:** store each user's edited checklist and completion state without building a separate account service; use a free development tier while its limits fit the pilot.
- **Google Gemini API, Gemini 2.5 Flash-Lite:** start with an inexpensive model for document-to-checklist extraction and retain it only if evaluation supports the choice. Standard paid text rates are \$0.10 per million input tokens and \$0.40 per million output tokens.
- **Vitest and Playwright:** check quote/page validation and the complete upload, edit, save, and export flow against reproducible fixtures.
- **Vercel:** host the Next.js demo with minimal deployment work; verify current plan and request limits before using its free development tier.

At 8,000 input tokens and 2,000 total output tokens, including any thinking tokens, one extraction costs about **\$0.0016**; 1,000 such calls cost **\$1.60**, excluding retries and hosting. Budget **\$10 for model usage** this semester, with an application-enforced call allowance and token limits. Measure actual usage during evaluation. Google pricing checked 5 Oct 2026: <https://ai.google.dev/gemini-api/docs/pricing#gemini-2.5-flash-lite>. Recheck model availability and prices before implementation.